

MixFormer: 工业推荐系统中稠密特征与序列的协同扩展

Xu Huang*
xu.hwang@outlook.com
ByteDance Inc.
上海, 中国

Hao Zhang*
zhanghao.229@bytedance.com
ByteDance Inc.
上海, 中国

Zhifang Fan*
fanzhifangfzf@gmail.com
ByteDance Inc.
上海, 中国

Yunwen Huang*
huangyunwen.eleanor@bytedance.com
ByteDance Inc.
北京, 中国

Zhuoxing Wei
weizhuoxing@bytedance.com
ByteDance Inc.
北京, 中国

Zheng Chai
chaizheng.cz@bytedance.com
ByteDance Inc.
杭州, 中国

Jinan Ni
nijinan@bytedance.com
ByteDance Inc.
上海, 中国

Yuchao Zheng
zhengyuchao.yc@bytedance.com
ByteDance Inc.
杭州, 中国

Qiwei Chen†
chenqiwei05@gmail.com
ByteDance Inc.
上海, 中国

摘要

随着工业推荐系统进入由规模化驱动的发展阶段, Transformer 架构因其能够将模型扩展至更大容量和更长序列而愈受到青睐。然而, 现有基于 Transformer 的推荐模型在结构上仍较为割裂, 其中序列建模与特征交互由参数相互独立的不同模块实现。这类设计带来了根本性的协同扩展难题: 在计算预算有限的情况下, 模型容量不得不在稠密特征交互与序列建模之间进行次优分配。在本工作中, 我们提出 **MixFormer**, 一种面向推荐系统定制的统一 Transformer 风格架构, 可在单一主干网络中联合建模序列行为与特征交互。通过统一参数化, MixFormer 能够在稠密容量和序列长度两个维度上实现有效的协同扩展, 从而缓解解耦设计中存在的权衡。此外, 一体化架构促进了序列表示与非序列表示之间的深度交互, 使高阶特征语义能够直接参与序列聚合, 并提升整体表达能力。为确保工业落地的可行性, 我们进一步提出用户—物品解耦策略以进行效率优化, 显著减少冗余计算和推理延迟。在大规模工业数据集上的广泛实验表明, MixFormer 在准确性和效率方面始终表现出优势。此外, 在 Douyin 和 Douyin Lite 两个生产级推荐系统上开展的大规模在线 A/B 测试显示, 包括活跃天数和应用内使用时长在内的用户参与度指标均获得稳定提升。

CCS 概念

• 信息系统 → 推荐系统.

关键词

推荐系统, 扩展定律, 用户序列建模

ACM 参考文献格式:

Xu Huang, Hao Zhang, Zhifang Fan, Yunwen Huang, Zhuoxing Wei, Zheng Chai, Jinan Ni, Yuchao Zheng, and Qiwei Chen. 2026.

*同等贡献。



This work is licensed under a Creative Commons Attribution 4.0 International License.

KDD '26, 韩国济州岛

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3818447>

MixFormer: 工业推荐系统中稠密特征与序列的协同扩展. In 第 32 届 ACM SIGKDD 知识发现与数据挖掘会议论文集 V.2 (KDD '26), 2026 年 8 月 09–13 日, 韩国济州岛. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3770855.3818447>

1 引言

Transformer 架构 [7, 15, 19] 已成为现代工业推荐系统的基础建模范式。得益于强大的表征能力和高度并行性, Transformer 尤其适合大规模用户行为建模与复杂特征交互学习。与此同时, 推荐系统已进入由规模扩展驱动的发展阶段, 性能提升越来越依赖数据量和模型容量的扩展, 而非手工设计的特征或特定于任务的启发式方法 [2, 6, 23–25, 29]。在这一阶段, 模型架构不再是次要的设计选择, 而是决定新增计算资源能否高效转化为可度量性能提升的首要因素。

现有 Transformer 在推荐系统中的应用大体沿着两个互补但基本相互独立的方向发展。一类工作聚焦序列建模, 利用 Transformer 捕获用户行为序列中的长程时间依赖 [2, 4, 5, 13, 14, 28]。另一类工作强调高阶特征交互, 采用 Transformer 风格的架构对异构稠密特征之间的复杂关系进行建模 [24, 25, 29]。尽管这两个方向各自都展现了显著的经验性能增益和良好的规模扩展趋势, 但它们隐含地假设序列建模与特征交互可以彼此独立地优化。随着推荐系统扩展到更长的用户历史、更丰富的特征空间和更大的模型容量, 这一假设变得愈发脆弱。

从系统层面看, 在有限计算预算下共同扩展序列建模和稠密特征交互, 会暴露出一种根本性的架构矛盾。序列 Transformer 的计算成本随序列长度迅速增长, 而稠密 Transformer 则主要随特征维度和模型宽度扩展。当这两个组件分别进行参数化和优化时, 它们会争夺共同的计算与参数预算, 导致彼此冲突的扩展诉求。为序列组件分配更多容量可以改善时间建模, 却会不成比例地增加计算成本, 从而实质上抑制稠密特征交互的扩展潜力。反之, 优先扩展稠密部分又会限制模型利用长程行为信号的能力。因此, 对于强制实施严格参数分离的架构而言, 实现序列组件与稠密组件的全局最优协同扩展在结构上并不协调。

大多数现有方法通过分层堆叠或并行拼接, 将序列 Transformer 与稠密 Transformer 结合起来。在分层设计中, 序列 Transformer 的输出被视为稠密 Transformer 的输入特征 [2, 8, 29]; 而在并行设计中, 两类组件的输出会被拼接 [26]。尽管这些范式较为简单, 但它们仍严格分离序列参数与非序列参

数, 导致跨组件交互有限、优化过程割裂。更重要的是, 这种分离从根本上限制了协同扩展行为: 序列模块迅速增长的计算开销主导了资源分配决策, 使稠密组件无法按比例扩展, 并在现实部署约束下导致次优的全局性能。

本文认为, 大规模推荐系统要实现有效的协同扩展, 需要一种根本不同的架构原则: 必须在同一参数空间中统一序列建模与特征交互, 并在整个网络中对二者进行联合优化。为此, 我们提出 **MixFormer**, 这是一种完全统一的 Transformer 风格大型推荐模型, 它使用一组共享参数同时对序列交互和稠密特征交互进行建模。通过消除严格的参数边界, MixFormer 使序列与非序列的协同建模能够相互增强, 让高阶特征语义直接参与序列聚合, 同时保留细粒度行为信号。这种统一参数化从根本上解决了参数分配困境, 并为协同扩展建立了连贯的基础。

此外, 为使统一扩展在工业环境中切实可行, 我们引入一种用户—物品解耦策略, 利用请求级批处理 [8] 技术复用计算, 从而显著提升计算效率。该机制是在有限资源预算下扩展统一 Transformer 架构的关键支撑, 它有效提高了生产系统中可部署模型容量和序列长度的上限。

综上, 我们的贡献有三点:

- 我们提出 MixFormer, 一种面向推荐系统的完全统一 Transformer 架构, 它在单一参数空间内联合建模序列动态与稠密特征交互, 解决了现有混合设计的结构局限, 并实现有效的全局协同扩展。
- 我们提出一种用户—物品解耦策略, 用于请求级计算共享与削减, 使统一 Transformer 的扩展能够在大规模工业场景中落地。
- 在大规模工业数据集上的大量实验表明, MixFormer 在准确性和效率方面均表现更优, 并且随着模型容量和序列长度增加, 展现出更有利的协同扩展特性。

2 相关工作

2.1 序列建模

用户行为序列建模是推荐系统中的一个基础问题, 因为它能够捕捉用户动态变化的兴趣。早期且具有代表性的方法主要采用基于目标注意力的架构对用户近期行为进行建模, 包括 DIN [28]、DIEN [27] 和 BST [5]。这些方法通常通过关注有限窗口内的近期行为来强调用户的短期兴趣。然而, 如此短的序列往往不足以完整刻画用户的长期偏好。为解决这一局限, SIM [13] 引入了一种面向超长序列建模的检索式范式。具体而言, SIM 采用两阶段框架, 其中包括用于检索相关历史行为的通用搜索单元 (General Search Unit, GSU), 以及用于建模所检索子序列的精确搜索单元 (Extract Search Unit, ESU)。在此范式的基础上, 后续工作通过支持端到端训练 [4]、使用基于 BM25 的算法增强检索策略 [14], 以及利用一致性保持机制改进 GSU 模块 [3] 等方式 [1, 17, 22], 进一步优化了 SIM。随着基于 GPU 的硬件和大规模训练基础设施快速发展, 近期研究重新审视了端到端长序列建模。Longer [2] 提出层次化注意力架构, 以降低长序列自注意力的二次复杂度。与此同时, GR [23] 和 MTGR [11] 通过采用 Transformer 解码器风格的架构, 将点击率 (CTR) 预测重新表述为生成式任务。然而, 现有序列建模方法大多只关注序列信号, 可能导致查询表达能力不足并丢失细粒度行为信息。

2.2 特征交互

特征交互长期以来一直是推荐系统的核心研究主题, 旨在通过组合异构输入特征来构建高阶表示。因子分解机 [16] 是最早显

式建模二阶特征交互的方法之一; 此后, 大量扩展方法利用神经网络探索了高阶交互和隐式交叉特征 [9, 12, 20, 21]。然而, 近期研究表明, 许多现有神经交互模型难以有效扩展至工业规模的推荐场景 [24]。为应对这一挑战, WuKong [24] 被提出作为大规模特征交互主干网络, 并展现出强大的可扩展性和表达能力。沿着这一研究方向, RankMixer [29] 进一步提出了一种针对大规模工业排序系统定制的高效 Transformer 风格架构。尽管取得了成功, 这些特征交互方法通常仍将序列特征视为压缩表示或静态表示。因此, 交互模块产生的高阶特征与序列建模过程相互解耦, 限制了序列表示的表达能力。

工业推荐系统通常采用层次堆叠或并行组合范式来结合序列模块与非序列模块。近期, OneTrans [26] 尝试利用设计的注意力掩码和独立参数, 在 Transformer 主干网络中将序列特征与非序列特征建模为异构 token 序列, 从而统一二者。然而, 二次复杂度带来了严重的计算开销, 而相互分离的参数则造成了稠密容量与序列长度协同扩展的难题。

3 方法

3.1 整体架构

MixFormer 是一种面向多任务推荐系统定制的高效 decoder-only Transformer 变体, 由输入层和 L 个 MixFormer 块组成, 其后连接若干任务网络。输入层负责特征嵌入与拆分: 首先将用户、物品和上下文特征投影为嵌入向量, 再将该向量拆分为 N 个头。整体架构如图 1 所示。

3.2 特征嵌入与拆分

输入特征可以分为两类: 序列特征和非序列特征。

序列特征表示用户的历史交互序列, 由按时间顺序排列的用户物品行为组成。每个行为由物品标识符、行为类型、时间戳和一些可用的辅助信息描述。对于时间步 t 上的每个行为, 我们使用专用嵌入层对其组成特征进行嵌入, 并拼接所得向量以形成行为表征 s_t 。相应地, 长度为 T 的用户行为序列表示为

$$\mathbf{S} = [s_1, s_2, \dots, s_T],$$

并作为模型的序列输入。

非序列特征包括用户特征、物品特征和上下文特征。令 $\mathcal{F}_{\text{ns}} = \{f_1, f_2, \dots, f_M\}$ 表示非序列特征集合。每个特征 f_i 首先通过特征专属的嵌入表映射为稠密嵌入向量 $e_i \in \mathbb{R}^{d_i}$ 。随后将这些嵌入拼接成单一的复合表征:

$$\mathbf{e}_{\text{ns}} = [e_1; e_2; \dots; e_M] \in \mathbb{R}^{D_{\text{ns}}}, \quad (1)$$

其中, $D_{\text{ns}} = \sum_{i=1}^M d_i$ 表示所有非序列特征的总嵌入维度。之后, 非序列特征作为整个主干网络的查询输入。

在标准 Transformer 解码器中, 注意力模块 (自注意力或交叉注意力) 被扩展为多头形式, 其中查询被拆分到多个子空间中。为在 Transformer 风格架构中实现统一建模, 我们进一步将 $\mathbf{e}_{\text{ns}}$ 拆分为一组维度固定的特征头。具体而言, $\mathbf{e}_{\text{ns}}$ 被均匀划分为 N 个连续子向量, 每个子向量的维度为 $d = D_{\text{ns}}/N$ 。随后, 每个子向量被投影为一个 D 维向量:

$$\mathbf{x}_j = \mathbf{W}_j \cdot \mathbf{e}_{\text{ns}}[d \cdot (j-1) : d \cdot j], \quad j = 1, \dots, N, \quad (2)$$

其中, $\mathbf{W}_j \in \mathbb{R}^{D \times d}$ 表示可学习的线性矩阵, $\mathbf{x}_j \in \mathbb{R}^D$ 表示第 j 个非序列特征头。所得向量 $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{N \times D}$ 作为 MixFormer 块的非序列输入头。

与将所有特征压缩到单个头中相比, 把嵌入空间划分为多个头可以保留表征多样性, 使模型能够捕获异构特征语义, 而不会引入过多的结构复杂性。此外, 后续章节介绍的 MixFormer

块中的三个核心模块 (Query Mixer、Cross Attention 和 Output Fusion) 均采用多头方式设计。这种设计为下游模块提供了灵活且计算高效的接口, 使高阶特征交互能够在后续阶段与序列建模无缝融合。

3.3 MixFormer 块

MixFormer 采用在结构上与 Transformer 解码器对齐的 Transformer 风格架构。每个 MixFormer 块由三个核心模块组成: 查询混合器、交叉注意力和输出融合, 它们在概念上分别对应标准 Transformer 解码器块中的自注意力、交叉注意力和前馈网络 (FFN) 组件。该设计既保留了基于 Transformer 的架构的表达能力, 又支持面向大规模工业推荐系统定制的操作。

3.3.1 查询混合器. 自注意力在大语言模型中展现出卓越的有效性, 其中所有 token 均嵌入统一语义空间, 并且可以通过内积运算可靠地建模 token 之间的相似性。然而, 受 RankMixer [29] 启发, 在推荐场景中, 查询来源于高度异构的特征字段, 包括用户属性、物品属性和上下文信号。这些特征源自不同的语义空间, 并且通常对应规模极大且稀疏的 ID 域。在这种异构性下, 通过内积相似度计算注意力权重本身并不可靠, 因为不同特征空间之间难以建立有意义的对齐。因此, 自注意力不仅无法稳定提升建模效果, 还会引入大量计算开销。

因此, 我们使用受先前工作 [18, 29] 启发的轻量且硬件友好的查询混合器模块来替代自注意力。与多头自注意力模块类似, 查询混合器无需依赖基于相似度的注意力即可实现跨头信息交换, 因此更适合在严格的效率约束下进行异构特征建模。

形式化地, 给定输入查询 $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N] \in \mathbb{R}^{N \times D}$, $\mathbf{x}_i \in \mathbb{R}^d$ 。则查询混合器定义为:

$$\mathbf{P} = [\mathbf{p}_1, \dots, \mathbf{p}_N] = \text{HeadMixing}(\text{Norm}(\mathbf{X})) + \mathbf{X}, \quad (3)$$

$$\mathbf{q}_i = \text{SwiGLUFFN}_i(\text{Norm}(\mathbf{p}_i)) + \mathbf{p}_i. \quad (4)$$

这里, $\text{HeadMixing}(\cdot)$ 将 $\mathbf{X}'$ 重塑为 $\mathbb{R}^{N \times N \times \frac{D}{N}}$, 转置第一维和第二维, 并将结果重新展平为 $\mathbb{R}^{N \times D}$, 如图 1 所示。该操作以无参数的方式实现高效的跨头信息交换。值得注意的是, FFN 针对每个头独立实例化, 以显式考虑特征异构性, 称为逐头 FFN。先前工作 [29] 已证明, 该设计在保持良好效率的同时具备强大的表达能力。

3.3.2 交叉注意力. 尽管基于相似度的自注意力不适合建模异构非序列特征, 但它仍是将结构化查询表示与序列行为信号对齐的有效机制。在 MixFormer 中, 交叉注意力模块被设计为以查询混合器产生的高阶特征表示为条件来聚合用户序列。具体而言, 查询混合器输出的 N 个头被直接视为交叉注意力的 N 个头, 其中每个头都充当语义特化的子查询, 专注于非序列特征的不同子空间。该设计使用户偏好的不同方面能够以解耦但协调的方式关注行为序列, 无需额外的查询拆分投影矩阵, 并降低了头坍塌的风险。

我们首先使用每层独立且由 SwiGLU 激活的 FFN, 将行为序列中的每个动作转换为潜在向量 $\mathbf{h}_t$, 使其与查询输入对齐, 然后将其投影为键和值:

$$\mathbf{h}_t = \text{SwiGLUFFN}^{(l)}(\text{Norm}(\mathbf{s}_t)) + \mathbf{s}_t \in \mathbb{R}^{N^D}, \quad (5)$$

$$\mathbf{h}_t^i = \mathbf{h}_t[iD : (i+1)D] \in \mathbb{R}^D, \quad (6)$$

$$\mathbf{k}_t^i = \mathbf{W}_k^i \mathbf{h}_t^i, \quad \mathbf{v}_t = \mathbf{W}_v^i \mathbf{h}_t^i \quad (7)$$

其中, $\text{SwiGLU}^{(l)}$ 分别表示第 l 个 MixFormer 块中的 SwiGLUFFN。 $\mathbf{W}_k^i, \mathbf{W}_v^i \in \mathbb{R}^{D \times D}$ 表示第 i 个头的键和值投影矩阵。与 Transformer 解码器中依赖共享隐状态的标准交叉注意力机制不同, 这些逐层 SwiGLUFFN 在每一层均独立参数化, 使模型能够随深度增加逐步细化序列表示。

第 i 个查询头的输出计算如下:

$$\mathbf{z}_i = \sum_{t=1}^T \text{softmax}\left(\frac{\mathbf{q}_i^\top \mathbf{k}_t^i}{\sqrt{D}}\right) \mathbf{v}_t + \mathbf{q}_i, \quad i = 1, \dots, N. \quad (8)$$

得到的表示 $\{\mathbf{z}_1, \dots, \mathbf{z}_N\}$ 捕捉了以特征为条件的用户序列摘要, 并作为聚合后的序列输出用于后续融合。

3.3.3 输出融合. 在从查询混合器获得高阶非序列表示, 并从交叉注意力模块获得以特征为条件的序列聚合结果后, 输出融合层对这些信号进行深度整合, 以生成最终表示。

具体而言, 每个交叉注意力输出 $\mathbf{z}_i$ 同时包含非序列信息, 以及与相应高阶查询头对齐的序列信息。然而, 由于查询头具有异构性, 且先前模块引入了残差连接, 共享的前馈变换不足以充分捕捉各个头特有的交互。因此, 我们采用逐头、由 SwiGLU 激活的前馈网络, 进一步独立细化每个表示。

形式化地, 输出融合定义为:

$$\mathbf{o}_i = \text{SwiGLUFFN}_i(\text{Norm}(\mathbf{z}_i)) + \mathbf{z}_i, \quad (9)$$

其中, $\text{SwiGLUFFN}_i(\cdot)$ 表示头特定的 SwiGLUFFN。

该设计使 MixFormer 能够对序列信号与非序列信号进行深度非线性融合, 同时显式考虑特征异构性。通过对每个头应用独立变换, 模型能够保持头级别的特化, 并避免异构特征子空间之间的表示干扰, 从而获得表达能力更强且更稳定的表示。随后, 该输出作为下一个 MixFormer 块的输入, 实现渐进式的统一特征交互与序列建模。值得注意的是, 输出融合和查询混合器中的逐头 FFN 同时服务于序列特征与非序列特征, 形成统一参数化范式, 并解决两个独立模块组合时的参数分配难题。

3.4 用户—物品解耦

请求级批处理 (Request Level Batching, RLB) [8, 10] 是提升推荐训练和推理效率的一种高效范式, 它在单个请求内的多个候选目标之间共享用户侧计算, 从而大幅降低计算成本。然而, 原始统一 MixFormer 中混合的用户—物品计算限制了 RLB 的应用。为解决这一问题, 我们提出用户—物品解耦的 MixFormer 变体 (UI-MixFormer), 如图 2 所示。

3.4.1 特征解耦. 我们将非序列特征划分为互不相交的用户侧子集和物品侧子集, 并分别将其投影为 N_U 个头和 N_G 个头, 作为 Mixformer 的输入。为保持模型容量, 头的总数不变, 并根据嵌入维度计算 N_U 和 N_G : $N_G = \lfloor \frac{D_{ns}^G N}{D_{ns}} \rfloor$, $N_U = N - N_G$ (其中, D_{ns}^G 为物品侧非序列特征的嵌入维度)。在实践中, $N_U : N_G$ 设为 1:1。

3.4.2 带掩码的查询混合器. 由于 HeadMixing 操作作用于不同的头, 其输出不再保持纯粹的用户侧表示, 导致无法通过 RLB 进行加速。受自注意力中因果掩码的启发, 我们为 MixFormer 中从用户到物品的单向融合设计了一种掩码。如图 2 所示, 为确保用户侧的头能够在单次请求内复用, 我们使用如下定义的掩码矩阵 $\mathcal{M} \in \mathbb{R}^{N \times D}$, 去除用户侧头中的物品侧信

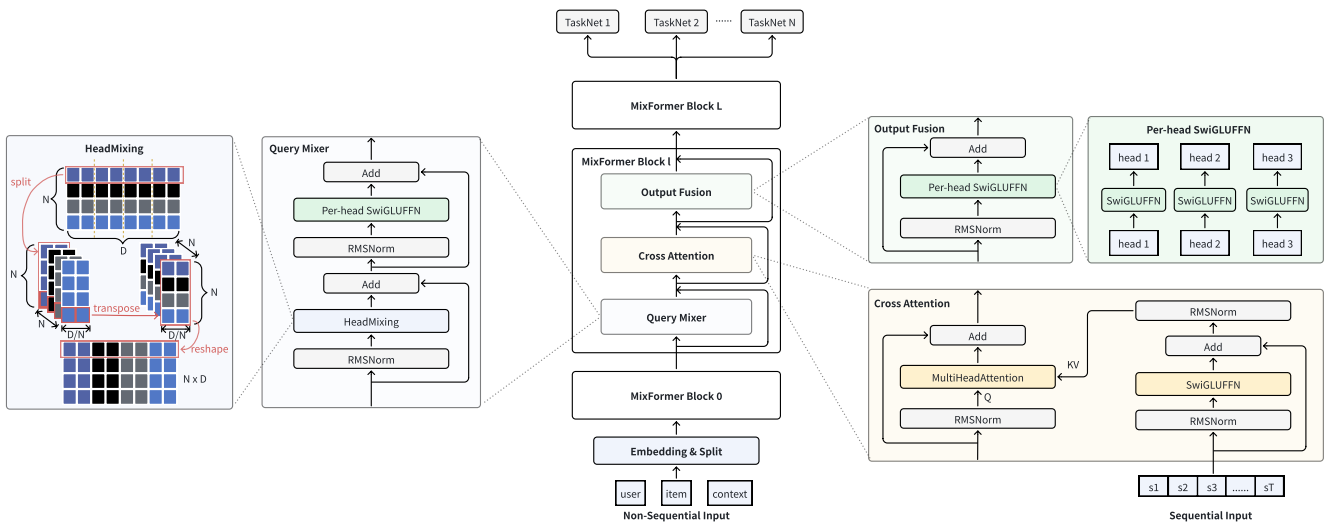


图 1: MixFormer 框架概览。该架构由特征嵌入与拆分层、随后堆叠的 L 个 MixFormer 块以及任务特定的头部组成。每个 MixFormer 块集成了三个核心组件: 查询混合器、交叉注意力和输出融合。

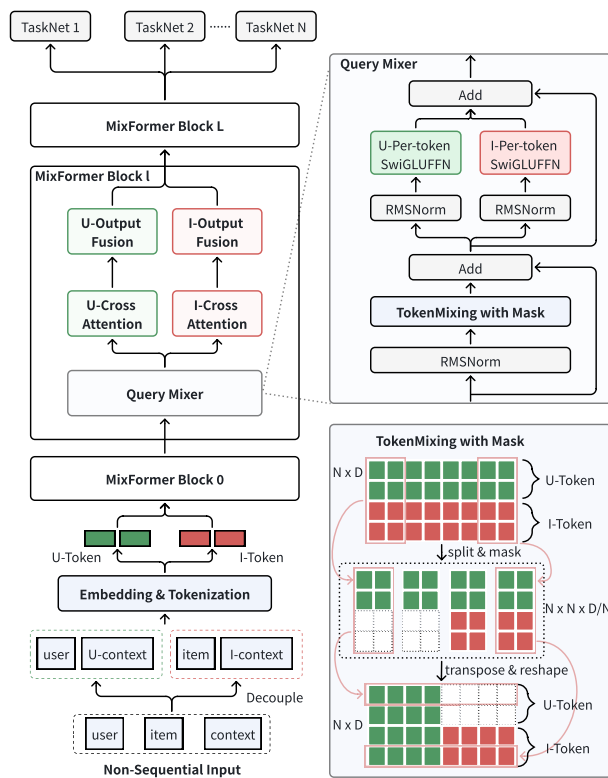


图 2: 采用请求级归约技术的用户-物品解耦 MixFormer 概览。绿色模块表示可在请求级共享的用户侧计算, 红色模块表示针对每个候选物品独立执行、因而无法共享的物品侧计算。

号:

$$\mathcal{M}[i, j] = \begin{cases} 0, & i < N_U \text{ and } j \geq N_U * \frac{D}{N} \\ 1, & \text{otherwise.} \end{cases} \quad (10)$$

最后, 将掩码矩阵 $\mathcal{M}$ 与原始 HeadMixing 操作的输出逐元素相乘, 得到用户-物品解耦的 HeadMixing 模块, 从而实现具有显式用户-物品解耦的查询混合器:

$$\text{HeadMixing}_{\text{decouple}}(\cdot) = \mathcal{M} \odot \text{HeadMixing}(\cdot) \quad (11)$$

与双塔架构不同,所提出的方法保留了从用户侧到物品侧的信息传播,从而确保能够有效建模用户-物品特征交互。值得注意的是,用户历史行为的序列特征同样适用于请求级共享。因此,用户侧头与序列之头的交叉注意力计算所产生的计算开销可以大幅降低。这一观察进一步凸显了统一建模范式优势:它将序列建模和特征交互整合到统一的主干网络中。

4 实验

4.1 实验设置

4.1.1 数据集. 我们在从抖音推荐系统收集的大规模离线数据集上进行实验。该数据集涵盖连续两周, 包含数万亿条用户-物品交互记录。每个样本均包含 300 多个特征, 大体可分为非序列特征和序列特征。非序列特征包括由用户画像、物品属性和上下文信息派生的类别特征、数值特征和交叉特征。序列特征对应用户历史, 其中每个行为由物品标识符、行为类型、时间戳和侧信息属性表示。

4.1.2 评估指标. 我们从准确性和效率两个方面评估模型性能。在准确性评估中，我们将任务表述为点击率 (CTR) 预测，这是一个二分类问题。我们采用 ROC 曲线下面积 (AUC) 和用户级 AUC (UAUC) 作为主要评估指标，因为它们在工业推荐系统中被广泛用于评估排序质量和用户级一致性。在效率评估中，我们报告模型参数量和浮点运算次数 (FLOPs)，它们可作为计算复杂度的代理指标，并反映模型在真实系统中的可扩展性和部署成本。

表 1: 与最先进基线在准确性和效率方面的性能对比 ($L = 512$)。粗体和下划线数字分别表示最佳和次佳结果。#Params 仅统计稠密参数, 包括输入投影和任务网络中的小型模块。为公平比较, 除 UI-MixFormer 外, FLOPs 均在未启用请求级批处理的情况下计算。“-”表示由于平台问题导致结果缺失。

类型	模型	完成		跳过		效率	
		AUC $\uparrow$	UAUC $\uparrow$	AUC $\uparrow$	UAUC $\uparrow$	# 参数量 (M)	GFLOPs/批次
堆叠式	TA $\rightarrow$ DLRM	0.8554	0.8270	0.8124	0.7294	9	52
	TA $\rightarrow$ DCNv2	+0.13%	+0.13%	+0.15%	+0.26%	22	170
	TA $\rightarrow$ DHEN	+0.18%	+0.26%	+0.36%	+0.52%	22	158
	TA $\rightarrow$ Wukong	+0.29%	+0.29%	+0.49%	+0.65%	122	442
	STCA $\rightarrow$ DCNv2	+0.89%	+0.91%	+1.05%	+1.65%	145	4,560
	TA $\rightarrow$ RankMixer	+0.95%	+1.22%	+1.25%	+1.82%	1,118	2,180
	STCA $\rightarrow$ RankMixer	+1.12%	+1.40%	+1.43%	+2.14%	1,255	6,736
并行式	OneTrans	+1.05%	+1.31%	+1.30%	+1.95%	316	23,371
	STCA $\oplus$ RankMixer	+1.11%	+1.38%	+1.42%	+2.11%	1,255	6,736
统一式	MixFormer-small	+1.01%	-	+1.18%	-	282	733
	MixFormer-medium	+1.28%	+1.60%	+1.60%	+2.46%	1,226	3,503
	UI-MixFormer-medium	+1.28%	+1.60%	+1.60%	+2.46%	1,226	2,242

4.1.3 基线. 我们将 MixFormer 与两类最先进的基线进行比较。(1) **堆叠式方法:** 首先将序列特征聚合为紧凑表示, 然后将其输入专用的特征交互模块。对于序列建模, 我们采用多个变体, 包括简单的目标注意力 (TA) 和最先进的堆叠目标交叉注意力 (STCA) 架构 [8]。对于特征交互, 我们考虑多种代表性方法, 包括 DLRM (原始 MLP)、DCNv2 [21]、Wukong [24]、DHEN [25] 和 RankMixer [29]。“A $\rightarrow$ B”表示将 A 的输出作为 B 的输入。(2) **并行式方法:** 并行拼接非序列模块和序列模块, 其中两个模块的参数相互独立。我们选择 RankMixer $\oplus$ STCA 和 OneTrans [26] 作为该类别的代表性基线。“A $\oplus$ B”表示将 A 和 B 的输出拼接后输入任务网络。对于 MixFormer, 我们实验了两种规模, 分别记为 MixFormer-small 和 MixFormer-medium。

4.1.4 实现细节. 所有实验均在数百块 GPU 上采用混合分布式训练框架完成, 其中稀疏部分异步更新, 而稠密部分同步更新。所有模型采用一致的优化器超参数。对于稠密部分, 我们使用学习率为 0.01 的 RMSProp 优化器, 而稀疏部分使用 Adam 优化器。所有实验的批次大小均设为 1,500。MixFormer-small 和 MixFormer-medium 的超参数分别设置为 $N = 16, L = 4, D = 386$ 和 $N = 16, L = 4, D = 768$ 。

4.2 与基线的比较

为展示所提出统一模型在准确性和效率方面的性能, 我们将 MixFormer 与最先进的方法进行比较, 包括层次化堆叠方法和两种代表性方法。结果如表 1 所示, 据此我们得到以下发现:

在序列建模和特征交互方面, STCA 和 RankMixer 等最先进方法始终以明显优势超越 TA、DCN 和 Wukong 等其他架构。该结果从实证上验证了 Transformer 风格架构在捕获复杂依赖关系和建模大规模用户行为模式方面的有效性, 凸显了它们在工业推荐系统中的优势。

在可比的子结构配置下, 堆叠式与并行式设计范式之间的性能差异很小。这一观察表明, 当两个组件分别参数化时, 序列建模与特征交互之间的交互仍然相对浅层。因此, 仅改变连接模式不足以带来显著的性能提升, 这说明需要对两个模块进行更深层次的整合, 才能充分发挥它们的互补优势。

得益于统一架构, 所提出的 MixFormer 始终优于所有基线方法。提升主要源于特征交互和序列建模组件之间的参数共享, 这促进了更强的跨模块表示学习。值得注意的是, 在参数预算与最强基线 STCA + RankMixer 相当时, MixFormer 在两个任务上均取得了显著的 AUC 提升。此外, 采用用户-物品解耦的效率优化变体在性能几乎不变的同时大幅降低了计算开销, 实现了 $\sim 36\%$ 的 FLOPs 降幅。

4.3 消融实验

为验证 MixFormer 中各个所提模块的有效性, 我们通过替换或删除部分设计进行了一系列实验, 具体如下:

- [QM] wo HM: 删除查询混合器中的 HeadMixing;
- [QM] HM $\rightarrow$ SA: 用 SelfAttention 模块替换查询混合器中的 HeadMixing;
- [QM] wo FFN: 删除查询混合器中的逐头 FFN;
- [CA] PL-FFN $\rightarrow$ FFN: 用共享 FFN 替换交叉注意力中用于行为投影的逐层 FFN。
- [OF] PFFN $\rightarrow$ FFN: 用各头共享的 SwiGLUFFN 替换输出融合中的逐头 SwiGLUFFN;
- Pre-RN $\rightarrow$ Post-LN: 将每个块中的前置 RMSNorm 替换为后置 LayerNorm;

消融结果如图 3 所示。结果表明, HeadMixing 和查询混合器中的逐头 FFN 都对性能提升做出了显著贡献, 这突显了在为序列建模构建查询时高阶特征表示的重要性。值得注意的是, 将 HeadMixing 替换为计算成本更高的自注意力并未带来可观察的性能增益, 这从实证上验证了零成本 HeadMixing 在处理推荐场景中的异构头时的效率和有效性。此外, 结果表明, 在交叉注意力块中, 使用逐层 FFN 能够在不同层生成差异化程度更高的表示, 在不增加计算成本的情况下提升模型表达能力。在输出侧, 将标准 FFN 升级为逐头 FFN, 可在不增加 FLOPs 的情况下进一步增强模型对异构头的表达能力, 并带来明显的性能提升。总体而言, 这些结果表明, 针对推荐系统提出的改进在有效性和效率两方面都取得了良好提升。

表 2: 抖音和抖音极速版信息流推荐场景的在线 A/B 测试结果。提升幅度是相对于线上部署的最具竞争力的 STCA→RankMixer 基线计算的。所有报告的提升均具有统计显著性, 且尚未收敛; 此处展示的结果仍在持续改善。

	Douyin app					Douyin Lite				
	活跃天数 ↑	时长 ↑	点赞 ↑	完播 ↑	评论 ↑	活跃天数 ↑	时长 ↑	点赞 ↑	完播 ↑	评论 ↑
总体	+0.0415%	+0.2799%	+0.1766%	+0.3897%	+0.7035%	+0.0252%	+0.4105%	+0.2125%	+0.2924%	+1.9097%
低活跃	+0.2263%	+0.2468%	+0.0771%	+0.4123%	+1.2483%	+0.2543%	+0.6044%	+3.0565%	+0.6157%	+2.6452%
中活跃	+0.0998%	+0.2719%	+0.2445%	+0.2796%	+0.6718%	+0.1218%	+0.4184%	+0.2329%	+0.2951%	+1.3286%
高活跃	+0.0203%	+0.2938%	+0.381%	+0.3335%	+0.8356%	+0.0237%	+0.4042%	+0.4871%	+0.2097%	+2.117%

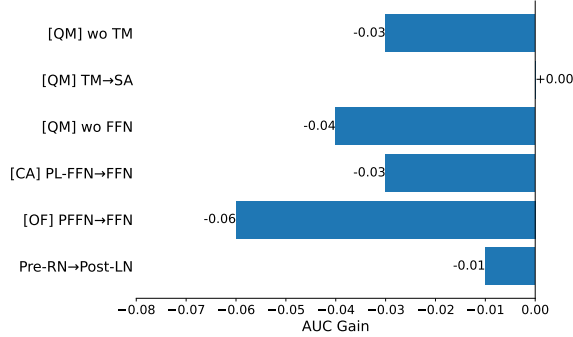


图 3: MixFormer 各模块的消融实验。AUC 增益以所提出的 MixFormer-small 为参照。QM、CA、OF、HM 和 SA 分别是 Query Mixer, Cross Attention, Output Fusion, HeadMixing 和 SelfAttention 的缩写。PT-FFN 和 PFFN 分别表示逐层 FFN 和逐头 FFN。

4.4 扩展分析

为评估所提模型的协同扩展能力, 我们分别在输入固定时扩展稠密参数, 以及在模型规模固定时增加序列长度, 并评估其性能。

4.4.1 稠密部分的扩展. 由于序列建模的计算成本对序列长度高度敏感, 仅根据参数量比较稠密扩展行为对非序列模型并不公平。在实践中, 服务成本主要由 FLOPs 决定; 因此, 图 4 给出了 AUC 增益随 FLOPs 变化的结果。我们比较了最先进的序列模型 STCA (配备轻量级 DCNv2 头)、非序列 RankMixer 模型 (配备单层 Target Attention), 以及二者按 1:1 FLOPs 组合的模型。在序列长度固定的设置下, 扩展 RankMixer 所带来的边际 AUC 增益大于扩展序列组件, 突显了当序列长度固定时目标物品特征交互的重要性。组合基线则在二者之间呈现出明显的权衡。相比之下, MixFormer 凭借统一参数化以及序列组件与非序列组件之间的深度交互, 实现了更高的截距和具有竞争力的扩展斜率。因此, 在不同 FLOPs 预算下, MixFormer 始终优于其他设计。

4.4.2 序列扩展性. 从另一个角度出发, 我们在固定稠密参数预算下考察不同方法随序列长度扩展的表现。为确保公平性, 我们选择了若干在序列长度为 512 时 FLOPs 相当的模型配置, 并在图 4 中用圆圈标出。随后, 我们按照 [8] 扩展输入序列长度, 评估长度为 {512, 2,048, 8,192, 10,000} 的行为序列。结果如图 5 所示。有趣的是, 随序列长度变化的扩展趋势与稠密扩展呈现相反模式。STCA 等将更多计算分配给序列组件的序列模型, 从更长序列中获益的程度显著高于非序列模型, 这与 [8] 中的观察一致。值得注意的是, 得益于序列组件和非序列

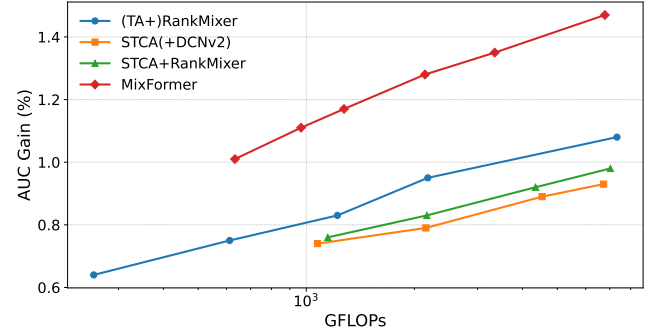


图 4: 基于 FLOPs 的扩展研究。序列长度固定为 512。(A+)B 表示模块 A 的规模保持固定, 而模块 B 的规模进行扩展。

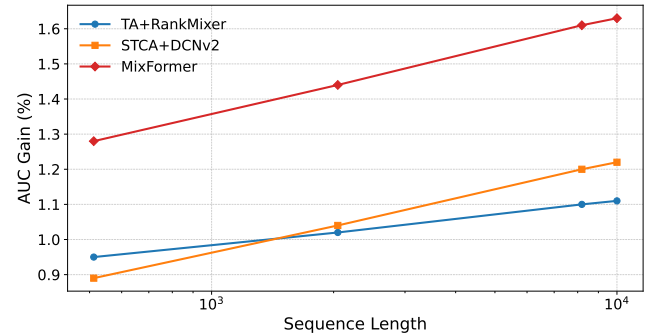


图 5: 关于序列长度的扩展研究。序列长度在 {512, 2048, 8192, 10000} 中扩展。

组件间的统一参数化, MixFormer 实现了与最先进的 STCA 相当的扩展斜率。

总体而言, 我们观察到, 序列组件和非序列组件独立参数化的模型面临明显的协同扩展权衡。在有限的计算预算下, 这类模型必须在 FLOPs 效率更高的非序列模块与更受益于序列长度扩展的序列模块之间谨慎分配容量。相比之下, 凭借集成式参数设计, MixFormer 在稠密扩展和序列扩展两方面均表现出最先进的扩展能力, 进一步验证了该架构的有效性。

4.5 服务延迟分析

我们还开展了服务延迟测试, 以验证用户—物品解耦策略的效率。如图 6 所示, 与 Request Level Batching 结合后, 用户—物

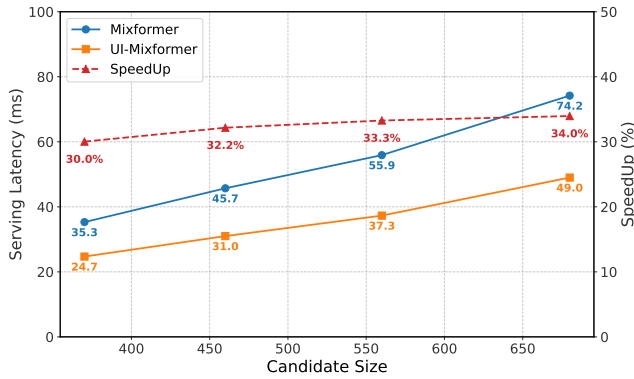


图 6: 不同候选规模下 Mixformer 与用户—物品解耦 Mixformer (UI-Mixformer) 的服务延迟和推理加速比。

品解耦的 MixFormer 实现了超过 30% 的服务速度提升。此外, 随着排序阶段待预测候选物品数量的增加, GPU 利用率逐渐趋于饱和, 从而进一步加剧计算瓶颈。所幸, 我们的用户—物品解耦设计能够在单次请求内的多个候选选项之间共享用户侧计算。因此, 相较于原始模型, 解耦后的 MixFormer 延迟增长明显更为平缓, 提升了候选集较大时的服务效率。

4.6 在线 A/B 测试

为验证 MixFormer 作为协同扩展推荐模型框架的通用性, 我们在两个 App 的个性化排序核心场景——信息流推荐中开展了在线实验。对于信息流推荐, 我们监测以下关键性能指标: 活跃天数指实验期间每位用户的平均活跃天数, 是 DAU 增长的替代指标; 时长衡量用户在 App 中的累计停留时间; 播完/点赞/评论/

户的完整播放、点赞和评论行为。

我们开展在线 A/B 测试实验, 与此前的基线进行比较: 参数量超过 1B 的堆叠式 STCA→RankMixer。两周内两个 App 信息流推荐的 A/B 测试结果如表 2 所示, 且提升幅度仍在增长, 表明收益尚未饱和。

5 结论

本文提出了一种面向推荐系统的新型 Transformer 风格统一架构。该模型将现代推荐系统的两个核心组件——序列建模与特征交互——整合进统一的统一参数化框架中, 从而形成高效且易于协同扩展的设计。此外, 我们还引入了显著的用户—物品解耦优化, 明确利用请求级计算削减来提升计算效率。

大量实验表明, 与强基线相比, 所提出的架构始终能够实现更优的效果和效率。我们进一步分析了模型随 FLOPs 和序列长度变化的扩展效应, 并在工业环境中开展了大规模在线 A/B 测试, 为该模型的协同扩展潜力及其在现实推荐系统中的实际有效性提供了有力证据。

参考文献

- [1] Yue Cao, Xiaojiang Zhou, Jiaqi Feng, Peihao Huang, Yao Xiao, Dayao Chen, and Sheng Chen. 2022. Sampling Is All You Need on Modeling Long-Term User Behaviors for CTR Prediction. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (Atlanta, GA, USA) (CIKM '22)*. Association for Computing Machinery, New York, NY, USA, 2974–2983. doi:10.1145/3511808.3557082
- [2] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, Xionghang Xie, Shiru Ren, Xiang Sun, Yaocheng Tan, Peng Xu, Yuchao Zheng, and Di Wu. 2025. LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25)*. Association for Computing Machinery, New York, NY, USA, 247–256. doi:10.1145/3705328.3748065
- [3] Jianxin Chang, Chenbin Zhang, Zhiyi Fu, Xiaoxue Zang, Lin Guan, Jing Lu, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. TWIN: Two-stage Interest Network for Life-long User Behavior Modeling in CTR Prediction at Kuaishou. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Long Beach, CA, USA) (KDD '23)*. Association for Computing Machinery, New York, NY, USA, 3785–3794. doi:10.1145/3580305.3599922
- [4] Qiwei Chen, Changhua Pei, Shanshan Lv, Chao Li, Junfeng Ge, and Wenwu Ou. 2021. End-to-End User Behavior Retrieval in Click-Through Rate Prediction Model. *CoRR* abs/2108.04468 (2021). arXiv:2108.04468 <https://arxiv.org/abs/2108.04468>
- [5] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in Alibaba. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data (Anchorage, Alaska) (DLP-KDD '19)*. Association for Computing Machinery, New York, NY, USA, Article 12, 4 pages. doi:10.1145/3326937.3341261
- [6] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. arXiv:2502.18965 [cs.LG] <https://arxiv.org/abs/2502.18965>
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. doi:10.18653/v1/N19-1423
- [8] Lin Guan, Jia-Qi Yang, Zhishan Zhao, Beichuan Zhang, Bo Sun, Xuanyuan Luo, Jinan Ni, Xiaowen Li, Yuhang Qi, Zhifang Fan, Hangyu Wang, Qiwei Chen, Yi Cheng, Feng Zhang, and Xiao Yang. 2026. Make It Long, Keep It Fast: End-to-End 10k-Sequence Modeling at Billion Scale on Douyin. In *Proceedings of the ACM Web Conference 2026, WWW 2026, Dubai, United Arab Emirates, originally scheduled for April 13-17, 2026, rescheduled for June 29 - July 3, 2026*, Hakim Hacid, Yoelle Maarek, Francesco Bonchi, Ido Guy, and Emine Yilmaz (Eds.). ACM, 7989–7998. doi:10.1145/3774904.3792811
- [9] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiquang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (Melbourne, Australia) (IJCAI'17)*. AAAI Press, 1725–1731.
- [10] Liang Guo, Wei Li, Lucy Liao, Huihui Cheng, Rui Zhang, Yu Shi, Yueming Wang, Yanzun Huang, Keke Zhai, Pengchao Wang, Timothy Shi, Xuan Cao, Shengzhi Wang, Renqin Cai, Zhaojie Gong, Omkar Vichare, Rui Jian, Leon Gao, Shiyuan Deng, Xingyu Liu, Xiong Zhang, Fu Li, Wenlei Xie, Bin Wen, Rui Li, Lu Fang, Xing Liu, and Jiaqi Zhai. 2025. Request-Only Optimization for Recommendation Systems. arXiv:2508.05640 [cs.LG] <https://arxiv.org/abs/2508.05640>
- [11] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang, Xiaoguang Li, Chunzhen Jing, Yueming Han, Menglei Zhou, Lei Yu, Chuan Liu, and

- Wei Lin. 2025. MTGR: Industrial-Scale Generative Recommendation Framework in Meituan. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management* (Seoul, Republic of Korea) (CIKM '25). Association for Computing Machinery, New York, NY, USA, 5731–5738. doi:10.1145/3746252.3761565
- [12] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xDeepFM: Combining Explicit and Implicit Feature Interactions for Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) (KDD '18). Association for Computing Machinery, New York, NY, USA, 1754–1763. doi:10.1145/3219819.3220023
- [13] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management* (Virtual Event, Ireland) (CIKM '20). Association for Computing Machinery, New York, NY, USA, 2685–2692. doi:10.1145/3340531.3412744
- [14] Jiarui Qin, Weinan Zhang, Rong Su, Zhirong Liu, Weiwen Liu, Guangpeng Zhao, Hao Li, Ruiming Tang, Xiuqiang He, and Yong Yu. 2023. Learning to Retrieve User Behaviors for Click-through Rate Estimation. *ACM Trans. Inf. Syst.* 41, 4, Article 98 (April 2023), 31 pages. doi:10.1145/3579354
- [15] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [16] Steffen Rendle. 2010. Factorization Machines. In *2010 IEEE International Conference on Data Mining*. 995–1000. doi:10.1109/ICDM.2010.127
- [17] Zihua Si, Lin Guan, Zhongxiang Sun, Xiaoxue Zang, Jing Lu, Yiqun Hui, Xingchao Cao, Zeyu Yang, Yichen Zheng, Dewei Leng, Kai Zheng, Chenbin Zhang, Yanan Niu, Yang Song, and Kun Gai. 2024. TWIN V2: Scaling Ultra-Long User Behavior Sequence Modeling for Enhanced CTR Prediction at Kuaishou. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, USA) (CIKM '24). Association for Computing Machinery, New York, NY, USA, 4890–4897. doi:10.1145/3627673.3680030
- [18] Ilya Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, Mario Lucic, and Alexey Dosovitskiy. 2021. MLP-mixer: an all-MLP architecture for vision. In *Proceedings of the 35th International Conference on Neural Information Processing Systems* (NIPS '21). Curran Associates Inc., Red Hook, NY, USA, Article 1857, 12 pages.
- [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [20] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. In *Proceedings of the ADKDD'17* (Halifax, NS, Canada) (ADKDD'17). Association for Computing Machinery, New York, NY, USA, Article 12, 7 pages. doi:10.1145/3124749.3124754
- [21] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of the Web Conference 2021* (Ljubljana, Slovenia) (WWW '21). Association for Computing Machinery, New York, NY, USA, 1785–1797. doi:10.1145/3442381.3450078
- [22] Xiang Xu, Hao Wang, Wei Guo, Luankang Zhang, Wanshan Yang, Runlong Yu, Yong Liu, Defu Lian, and Enhong Chen. 2025. Multi-granularity Interest Retrieval and Refinement Network for Long-Term User Behavior Modeling in CTR Prediction. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1* (Toronto ON, Canada) (KDD '25). Association for Computing Machinery, New York, NY, USA, 2745–2755. doi:10.1145/3690624.3709438
- [23] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Proceedings of the 41st International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 235), Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 58484–58509. <https://proceedings.mlr.press/v235/zhai24a.html>
- [24] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Shen Li, Yanli Zhao, Yuchen Hao, Yantao Yao, Ellie Dingqiao Wen, Jongsoo Park, Maxim Naumov, and Wenlin Chen. 2024. Wukong: towards a scaling law for large-scale recommendation. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (ICML'24). JMLR.org, Article 2455, 14 pages.
- [25] Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, Yang Liu, Huayu Li, Yasmine Badr, Jongsoo Park, Jiyan Yang, Dheevatsa Mudigere, and Ellie Wen. 2022. DHEN: A Deep and Hierarchical Ensemble Network for Large-Scale Click-Through Rate Prediction. arXiv:2203.11014 [cs.LR] <https://arxiv.org/abs/2203.11014>
- [26] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2025. OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender. arXiv:2510.26104 [cs.LR] <https://arxiv.org/abs/2510.26104>
- [27] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence* (Honolulu, Hawaii, USA) (AAAI'19/IAAI'19/EAAI'19). AAAI Press, Article 729, 8 pages. doi:10.1609/aaai.v33i01.33015941
- [28] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) (KDD '18). Association for Computing Machinery, New York, NY, USA, 1059–1068. doi:10.1145/3219819.3219823
- [29] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, Huizhi Yang, Zheng Chai, Zhe Chen, Yuchao Zheng, Qiwei Chen, Feng Zhang, Xun Zhou, Peng Xu, Xiao Yang, Di Wu, and Zuotao Liu. 2025. RankMixer: Scaling Up Ranking Models in Industrial Recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management* (Seoul, Republic of Korea) (CIKM '25). Association for Computing Machinery, New York, NY, USA, 6309–6316. doi:10.1145/3746252.3761507